Audited by CRED SHIELDS

CredShields

Smart Contract Audit

March 26, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of PropDefi between February 28, 2026, and March 3, 2026. A retest was performed on March 11, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

PropDefi

AICPA SOC | SCS Advocates OWASP OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

1. Executive Summary ——————————- **3**

State of Security 4

2. The Methodology ——————————- **5**

2.1 Preparation Phase 5

2.1.1 Scope 5

2.1.2 Documentation 6

2.1.3 Audit Goals 6

2.2 Retesting Phase 6

2.3 Vulnerability classification and severity 7

2.4 CredShields staff 8

3. Findings Summary ——————————- **10**

3.1 Findings Overview 10

3.1.1 Vulnerability Summary 10

5. Bug Reports —————————— **12**

Bug ID #C001 [Fixed] 12

Incorrect USDT Decimal Handling 12

Bug ID #H001 [Fixed] 13

Signature Malleability in Ecrecover 13

Bug ID #M001 [Fixed] 14

Missing Price Feed Validation 14

Bug ID #M002 [Fixed] 15

Stuck Native Funds When Paying with USDT 15

Bug ID #L001 [Fixed] 16

Floating and Outdated Pragma 16

Bug ID #I001 [Acknowledged] 17

Improper Nonce Management 17

Bug ID #I002 [Not Fixed] 18

Use Ownable2Step 18

Bug ID #I003 [Fixed] 19

Missing zero address validations 19

Bug ID #G001 [Fixed] 20

Large Number Literals 20

6. The Disclosure ——————————- **21**

1. Executive Summary ----------

PropDefi engaged CredShields to perform a smart contract audit from February 28, 2026, to March 3, 2026. During this timeframe, 9 vulnerabilities were identified. **A retest was performed on March 11, 2026, and all the bugs have been addressed.**

During the audit, 2 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "PropDefi" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Token and ICO Contracts	1	1	2	1	3	1	**9**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Token and ICO Contract's scope during the testing window while abiding by the policies set forth by PropDefi's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both PropDefi's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at PropDefi can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, PropDefi can future-proof its security posture and protect its assets.

2. The Methodology ----------

PropDefi engaged CredShields to perform a Token and ICO Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from February 28, 2026, to March 3, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Audited Scope: • https://testnet.bscscan.com/address/0x75b04dD4D091e965A0c8438a1E2334FcB2CD2E9d#code • https://testnet.bscscan.com/address/0x46a37fd18568df947af08e547376508beaea1a70#code **Retested Scope:** • https://testnet.bscscan.com/address/0x5BB373697cee7Ea0fB183E97913e543f2efD335e#code • https://testnet.bscscan.com/address/0x1D55046f1950ee9372880AE5e174c9D48bF08Eab#code

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

PropDefi is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 9 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Incorrect USDT Decimal Handling	Critical	Business Logic (SC03-LogicErrors)	Fixed
Signature Malleability in Ecrecover	High	Signature Replay Attacks (SCWE-055)	Fixed
Missing Price Feed Validation	Medium	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
Stuck Native Funds When Paying with USDT	Medium	Business Logic (SC03-LogicErrors)	Fixed
Floating and Outdated Pragma	Low	Floating Pragma (SCWE-060)	Fixed
Improper Nonce Management	Informational	Business Logic (SC03-LogicErrors)	Acknowledged
Use Ownable2Step	Informational	Missing Best Practices	Not Fixed
Missing zero address validations	Informational	Missing Input Validation (SC04-Lack Of Input Validation)	Fixed

Large Number Literals	Gas	Gas Optimization (SCWE-082)	Fixed

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #C001 [Fixed]

Incorrect USDT Decimal Handling

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Critical

Description
The contract initializes paymentDetails[1] for USDT with decimal = 6, while on BNB Chain the deployed USDT token actually uses 18 decimals. In getToken, the code uses paymentDetails[paymentType].decimal to scale amounts: data = amount * tokenAmount / (10 ** paymentDetails[paymentType].decimal); Because the contract assumes 6 decimals instead of 18, all USDT-based calculations are off by 10^12 , severely mispricing token purchases. This means the number of ICO tokens issued per USDT (or the effective cost per token) will be drastically different from what the protocol designer expects. Depending on the direction of the error, users can either massively overpay or obtain far more tokens than they should. The issue is systemic: every interaction using paymentType = 1 (USDT) is affected

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L124

Impacts
Because of the wrong decimal configuration, the contract will compute token allocations with a factor error of 10^12 , leading either to enormous over-distribution of ICO tokens or effectively unusable pricing.

Remediation
It is recommended to use 10^18 as decimal for USDT in configuration.

Retest
This issue has been fixed by using 10^18 as decimal for USDT in configuration.

Bug ID #H001 [Fixed]

Signature Malleability in Ecrecover

Vulnerability Type
Signature Replay Attacks (SCWE-055)

Severity
High

Description
Signature Malleability is a vulnerability that can occur when improperly utilising ECDSA (Elliptic Curve Digital Signature Algorithm) signatures. The vulnerability allows an attacker to change the signature slightly without invalidating the signature itself. This often happens when a smart contract doesn't validate signatures properly, enabling attackers to modify them and potentially bypass security measures.

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L288

Impacts
Using a vulnerable version of ecrecover could allow users to use the signature signed by the same user twice by manipulating the signature such that it still stays valid. This could lead to a loss of funds.

Remediation
To fix the signature malleability vulnerability, follow these steps:

- Use a well-tested library like OpenZeppelin's ECDSA.sol that already implements a secure signature validation process.
- Make sure to enforce a specific condition on the s value during the signature validation process, ensuring it has a low value. This prevents attackers from manipulating the signature.

Retest
This issue has been fixed by using ECDSA for signatures.

Bug ID #M001 [Fixed]

Missing Price Feed Validation

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Medium

Description
Chainlink has a library AggregatorV3Interface with a function called latestRoundData(). This function returns the price feed among other details for the latest round.
The contract was found to be using latestRoundData() without proper input validations on the returned parameters which might result in a stale and outdated price.

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L161

Impacts
Having oracles with functions to fetch price feed without any validation might introduce erroneous or invalid price values that could result in an invalid price calculation further in the contract.

Remediation

It is recommended to have input validations for all the parameters obtained from the Chainlink price feed. Here's a sample implementation:

```
(uint80 roundID ,int256 price, uint256 timestamp, uint80 answeredInRound) = AggregatorV3Interface(chainLinkAggregatorMap[underlying]).latestRoundData();

require(answer > 0, "Chainlink price <= 0");
require(answeredInRound >= roundID, "Stale price");
require(timestamp != 0, "Round not complete");
```

Retest
Fixed in 0x1D55046f1950ee9372880AE5e174c9D48bF08Eab#L255

Bug ID #M002 [Fixed]

Stuck Native Funds When Paying with USDT

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
The buyToken function allows purchasing with either native currency or a payment token depending on paymentType. When paymentType is set to 1 (USDT purchase), the function still accepts msg.value but does not use or refund it. In this branch, only the USDT tokenAmount is processed and transferred via safeTransferFrom, while any native currency sent remains in the contract balance. There is no path for users to reclaim accidentally sent native funds when using the USDT path. This creates a situation where users who mistakenly send both USDT and native currency lose the native portion permanently. The issue stems from missing input validation and refund logic around msg.value in token-based payment flows.

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L172

Impacts
Users who call buyToken with paymentType = 1 and mistakenly send a non-zero msg.value will have their native currency trapped in the contract with no built-in withdrawal mechanism.

Remediation
When paymentType indicates a token purchase (e.g., USDT), explicitly enforce require(msg.value == 0, "Do not send native currency"); to prevent users from accidentally sending native funds.

Retest
Fixed in 0x1D55046f1950ee9372880AE5e174c9D48bF08Eab#L281

Bug ID #L001 [Fixed]

Floating and Outdated Pragma

Vulnerability Type
Floating Pragma (SCWE-060)

Severity
Low

Description
Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., ^0.8.20, 0.8.29. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L2
- 0x75b04dD4D091e965A0c8438a1E2334FcB2CD2E9d#code#F1#L10

Impacts
If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.
Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.
The likelihood of exploitation is low.

Remediation
Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.31 pragma version
Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/

Retest
This is fixed by updating the pragma version to 0.8.31.

Bug ID #I001 [Acknowledged]

Improper Nonce Management

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Informational

Description
The buyToken function relies on a user-supplied sign.nonce and only checks if that exact value was used before. Because the nonce is not enforced as a per-user, contract-controlled incremental value, the protocol cannot guarantee proper ordering or uniqueness of authorizations. This weak design makes it easier to misuse or replay signatures in ways not intended by the protocol, especially if off-chain nonce management is flawed.

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L180

Impacts
User supplied nonce can lead to vulnerabilities like gas griefing issues . According to best practices it is recommended to use incremented nonce.

Remediation
To fix this issue it is recommended to use incremented nonce.

Retest
Client's comment: On the report you have requested to increase the nonce value for each user, but on our application the nonce will be retrieved from the application end so that's not required, since its a unique value without repetition.

Bug ID #I002 [Not Fixed]

Use Ownable2Step

Vulnerability Type
Missing Best Practices

Severity
Informational

Description
The "Ownable2Step" pattern is an improvement over the traditional "Ownable" pattern, designed to enhance the security of ownership transfer functionality in a smart contract. Unlike the original "Ownable" pattern, where ownership can be transferred directly to a specified address, the "Ownable2Step" pattern introduces an additional step in the ownership transfer process. Ownership transfer only completes when the proposed new owner explicitly accepts the ownership, mitigating the risk of accidental or unintended ownership transfers to mistyped addresses.

Affected Code

- 0x75b04dD4D091e965A0c8438a1E2334FcB2CD2E9d#code#F1#L735

Impacts
Without the "Ownable2Step" pattern, the contract owner might inadvertently transfer ownership to an unintended or mistyped address, potentially leading to a loss of control over the contract. By adopting the "Ownable2Step" pattern, the smart contract becomes more resilient against external attacks aimed at seizing ownership or manipulating the contract's behavior.

Remediation
It is recommended to use either Ownable2Step or Ownable2StepUpgradeable depending on the smart contract.

Retest:
This issue is not fixed. Ownable is used instead of Ownable2Step or Ownable2StepUpgradeable.

Bug ID #I003 [Fixed]

Missing zero address validations

Vulnerability Type
Missing Input Validation (SC04-Lack Of Input Validation)

Severity
Informational

Description:
The contracts were found to be setting new addresses without proper validations for zero addresses.
Address type parameters should include a zero-address check otherwise contract functionality may become inaccessible or tokens burned forever.
Depending on the logic of the contract, this could prove fatal and the users or the contracts could lose their funds, or the ownership of the contract could be lost forever.

Affected Code

- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L104
- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L105
- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L106

Impacts
If address type parameters do not include a zero-address check, contract functionality may become unavailable or tokens may be burned permanently.

Remediation
Add a zero address validation to all the functions where addresses are being set.

Retest
This issue has been fixed by validating against zero address.

Bug ID #G001 [Fixed]

Large Number Literals

Vulnerability Type
Gas Optimization (SCWE-082)

Severity
Gas

Description
Solidity supports multiple rational and integer literals, including decimal fractions and scientific notations. The use of very large numbers with too many digits was detected in the code that could have been optimized using a different notation also supported by Solidity.

Affected Code

- 0x75b04dD4D091e965A0c8438a1E2334FcB2CD2E9d#code#F1#L736
- 0x46a37fd18568df947af08e547376508beaea1a70#code#F1#L65

Impacts
Having a large number literals in the code increases the gas usage of the contract during its deployment and when the functions are used or called from the contract.
It also makes the code harder to read and audit and increases the chances of introducing code errors.

Remediation
Scientific notation in the form of 2e10 is also supported, where the mantissa can be fractional, but the exponent has to be an integer. The literal MeE is equivalent to M * 10**E. Examples include 2e10, 2e10, 2e-10, 2.5e1, as suggested in the official solidity documentation. types.html#rational-and-integer-literals
It is recommended to use numbers in the form "35 * 1e7 * 1e18" or "35 * 1e25".
The numbers can also be represented by using underscores between them to make them more readable such as "35_00_00_000"

Retest
This issue has been fixed by using the recommended remediation.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com